



THE KNIFE

MANUALE TECNICO

Università degli Studi dell'Insubria – Laurea Triennale in Informatica

Progetto Laboratorio B

Creato da: Mattia Gerundino, Christian Pianarosa

SOMMARIO

INTRODUZIONE	2
LIBRERIE ESTERNE UTILIZZATE	2
USE CASE UML DIAGRAM	4
ARCHITETTURA DEL SISTEMA	5
Protocollo di comunicazione e stato della sessione	5
Modulo condiviso (DTO)	6
Modulo server	7
Modulo client.....	10
BASE DI DATI	12
Entità identificate	12
Relazioni e Cardinalità	12
Schema Concettuale ER (pre-ristrutturazione).....	0
Schema ER (post-ristrutturazione).....	0
Ristrutturazione dello Schema ER.....	1
Schema Relazionale	1
Vincoli e integrità dei dati.....	1
Utilizzo delle basi di dati nel sistema	2
STRUTTURE DATI.....	2
SCELTE ALGORITMICHE.....	3



INTRODUZIONE

THE KNIFE è un progetto realizzato per Laboratorio interdisciplinare A e B per il corso di laurea in informatica dell'Università degli Studi dell'Insubria anno 2025/2026.

È una piattaforma che consente di trovare ristoranti in tutto il mondo e selezionarli in base al luogo, alla tipologia del ristorante stesso, alla fascia di prezzo, alla possibilità o meno di prenotare un tavolo o di ordinare da asporto.

Questo documento descrive l'architettura tecnica dell'applicazione TheKnife, un sistema desktop sviluppato in Java 21 con interfaccia grafica JavaFX 21 e persistenza su database relazionale PostgreSQL. Il progetto è organizzato come un singolo modulo Maven (project.demo) che integra al proprio interno tre macro-componenti logiche ben distinte per responsabilità:

- un modulo client, costituito dalle viste FXML, dai relativi Controller JavaFX e dal componente di navigazione centralizzato (Navigator);
- un modulo server (nel senso di "livello di accesso ai dati"), costituito dalle classi del package db che incapsulano tutta la logica di connessione e interrogazione del database PostgreSQL tramite JDBC;
- un piccolo insieme di classi condivise (DTO/model), utilizzate sia dal client sia, indirettamente, dal livello di accesso ai dati per il trasporto delle informazioni tra i due strati.

Sebbene l'applicazione non sia un sistema client-server distribuito in senso stretto (client e "server" girano nello stesso processo Java, sulla stessa macchina), l'architettura del codice separa nettamente le responsabilità secondo un classico pattern a livelli (layered architecture): la UI non esegue mai query SQL direttamente, ma dialoga esclusivamente con un Data Access Object (DAO) che espone metodi ad alto livello (es. cercaRistorante(...), aggiungiPreferito(...), rispostaRecensione(...)).

LIBRERIE ESTERNE UTILIZZATE

Tutte le dipendenze del progetto sono dichiarate nel file pom.xml e gestite tramite Apache Maven. La tabella seguente riassume le librerie esterne utilizzate, il loro ruolo nell'applicazione e la versione dichiarata.

Libreria	Versione	Ruolo nel progetto
org.postgresql:postgresql	42.7.3	Driver JDBC ufficiale per la connessione e l'interrogazione del database PostgreSQL da parte del livello DAO (package db).
org.openjfx:javafx-controls	21	Componenti grafici di base di JavaFX (Button, Label, TextField, ComboBox, TableView, ecc.) usati in tutte le viste dell'applicazione.
org.openjfx:javafx-fxml	21	Motore di caricamento e binding dei file FXML tramite FXMLLoader, con associazione automatica ai Controller annotati con @FXML.
org.openjfx:javafx-web	21	Motore WebView di JavaFX, incluso tra le dipendenze per eventuali visualizzazioni HTML/mappe embedded.
org.openjfx:javafx-swing	21	Livello di interoperabilità JavaFX/Swing, incluso come dipendenza di supporto.



Libreria	Versione	Ruolo nel progetto
org.openjfx:javafx-media	21	Supporto alla riproduzione di contenuti multimediali (audio/video) in JavaFX.
org.controlsfx:controlsfx	11.2.1	Libreria di componenti UI aggiuntivi per JavaFX (es. notifiche, controlli avanzati) non disponibili nel core.
com.dlsc.formsfx:formsfx-core	11.6.0	Framework per la costruzione dichiarativa di moduli/form complessi in JavaFX.
net.synedra:validatorfx	0.5.0	Libreria per la validazione dei campi di input nei form (es. controlli di formato su email, numeri, date).
org.kordamp.ikonli:ikonli-javafx	12.3.1	Set di icone vettoriali integrabili nei controlli JavaFX.
org.kordamp.bootstrapfx:bootstrapfx-core	0.4.0	Foglio di stile ispirato a Bootstrap per JavaFX, utilizzabile come base per lo stile dei controlli.
eu.hansolo:tilesfx	21.0.3	Libreria di componenti "tile" per dashboard e visualizzazioni sintetiche di dati (KPI, indicatori).
com.github.almasb:fxgl	17.3	Game engine/framework JavaFX incluso tra le dipendenze, potenzialmente utile per animazioni e interazioni avanzate dell'interfaccia.
org.junit.jupiter:junit-jupiter-api/-engine	5.10.2	Framework di unit testing (JUnit 5), dipendenza di scope test.

Le principali dipendenze di build (plugin Maven) sono invece:

Plugin	Versione	Ruolo
maven-compiler-plugin	3.13.0	Configura il compilatore Java per il livello di linguaggio 21 (source/target).
javafx-maven-plugin	0.0.8	Permette di avviare l'applicazione JavaFX tramite mvn javafx:run e di produrre immagini jlink personalizzate (launcher app).

Nota architetturale: nonostante il ricco set di librerie UI disponibili (ControlsFX, FormsFX, ValidatorFX, Ikonli, BootstrapFX, TilesFX, FXGL), la maggior parte delle schermate attualmente implementate si basa sui controlli standard di javafx.controls e su un foglio di stile CSS personalizzato (style.css), che replica in JavaFX un linguaggio visivo simile a quello di un moderno sito web responsive (navbar, card, badge, chip, ecc.).

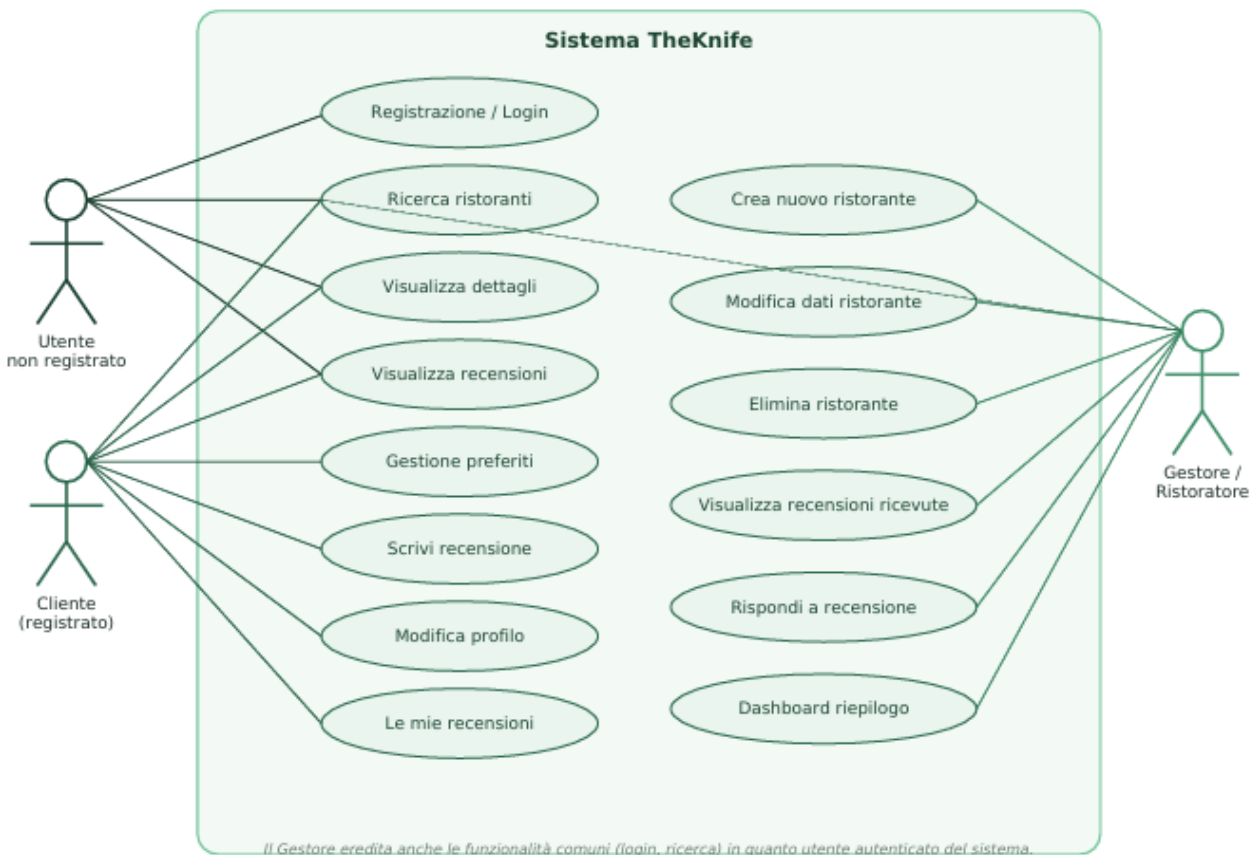


USE CASE UML DIAGRAM

Il diagramma seguente rappresenta i principali casi d'uso del sistema TheKnife e i tre attori che interagiscono con esso: l'Utente non registrato (visitatore anonimo), il Cliente (utente registrato con ruolo cliente nel database) e il Gestore/Ristoratore (utente registrato con ruolo gestore).

Attori e relativi casi d'uso principali:

- Utente non registrato: Registrazione/Login, Ricerca ristoranti, Visualizza dettagli, Visualizza recensioni
- Cliente (registrato): eredita i casi d'uso dell'utente non registrato e aggiunge Gestione preferiti, Scrivi recensione, Modifica profilo, Le mie recensioni
- Gestore/Ristoratore: eredita i casi d'uso comuni (login, ricerca) e aggiunge Crea nuovo ristorante, Modifica dati ristorante, Elimina ristorante, Visualizza recensioni ricevute, Rispondi a recensione, Dashboard riepilogo



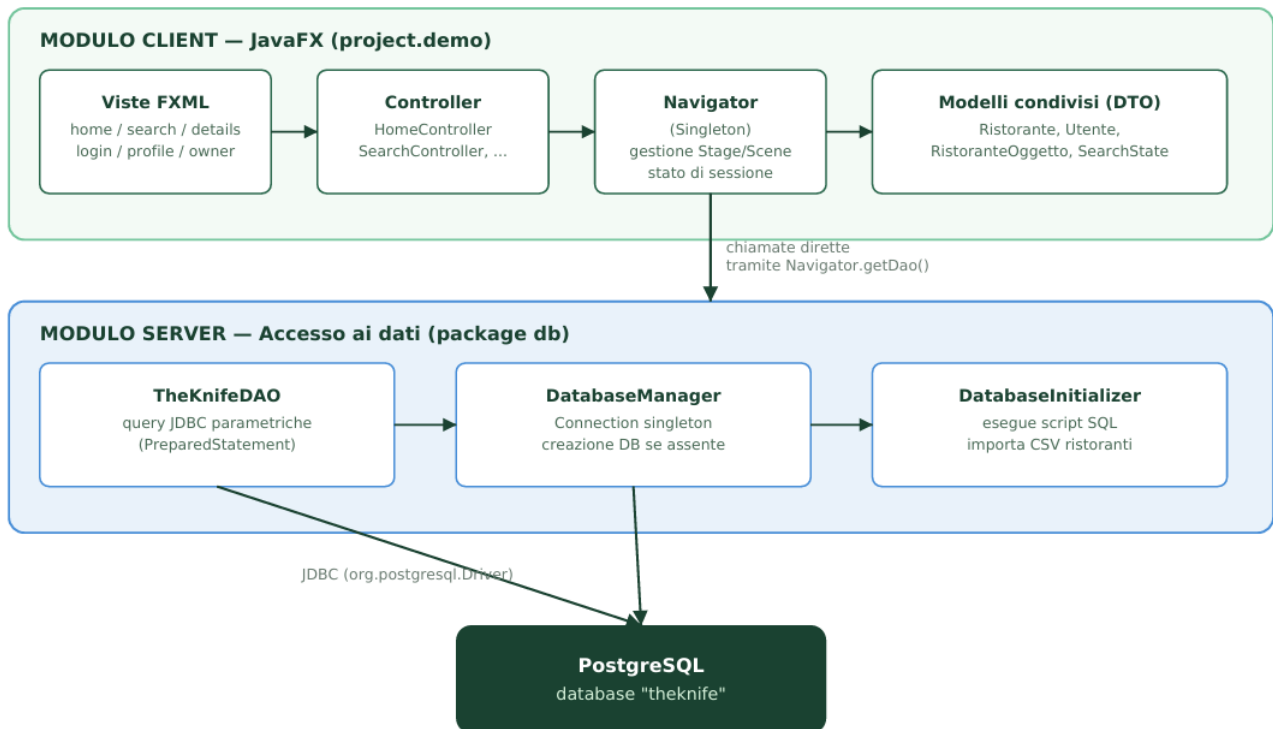
Le relazioni tra attori e casi d'uso rispecchiano fedelmente i vincoli implementati a livello di applicazione e di database:

- L'Utente non registrato può registrarsi/effettuare login, cercare ristoranti, visualizzare dettagli e recensioni già pubblicate, ma non può scrivere recensioni, salvare preferiti o gestire ristoranti.
- Il Cliente eredita tutte le funzionalità di consultazione e aggiunge: gestione dei preferiti, scrittura di recensioni, modifica del proprio profilo, consultazione delle proprie recensioni.
- Il Gestore dispone di casi d'uso esclusivi legati alla gestione dei ristoranti di sua proprietà: creazione, modifica ed eliminazione dei locali, consultazione delle recensioni ricevute e possibilità di rispondere pubblicamente a ciascuna di esse. Questi casi d'uso sono protetti sia a livello applicativo (i controller verificano il ruolo dell'utente loggato tramite `Navigator.getRuoloUtenteLoggato()`) sia a livello di database, tramite trigger dedicati.



ARCHITETTURA DEL SISTEMA

L'architettura di TheKnife segue un classico pattern a tre livelli (three-tier), pur essendo eseguita come singolo processo Java monolitico: modulo client (JavaFX), modulo server (accesso ai dati, package db) e database PostgreSQL, collegati tramite JDBC e raccordati dal Navigator, che espone alle viste sia lo Stage principale sia l'istanza condivisa del DAO.



Il Navigator (classe singleton) funge da elemento di raccordo tra i due macro-moduli: possiede un riferimento allo Stage principale di JavaFX (per il livello client) e un riferimento all'istanza di TheKnifeDAO collegata al database (per il livello server), esponendo entrambi ai vari Controller tramite i metodi getInstance(), getDao().

Protocollo di comunicazione e stato della sessione

Non essendo un'applicazione distribuita, TheKnife non utilizza un vero protocollo di rete (HTTP, socket TCP personalizzati, ecc.) per la comunicazione tra client e server: il "protocollo di comunicazione" tra i due livelli logici è costituito da chiamate a metodo dirette (invocazioni Java sincrone) verso l'istanza condivisa di TheKnifeDAO, la quale internamente traduce ogni richiesta in una query SQL parametrica eseguita tramite JDBC sulla connessione a PostgreSQL gestita da DatabaseManager.

Ogni operazione segue lo stesso schema concettuale:

- Un Controller JavaFX intercetta un evento dell'interfaccia (click su un pulsante, submit di un form, ecc.) tramite un handler annotato @FXML;
- Il Controller richiama Navigator.getInstance().getDao() per ottenere il riferimento al DAO condiviso;
- Viene invocato il metodo del DAO corrispondente all'operazione desiderata (es. cercaRistorante, aggiungiRecensione, rispostaRecensione), passando i parametri come argomenti tipizzati Java (String, Integer, Double, Boolean...);
- Il DAO costruisce ed esegue una PreparedStatement JDBC, restituendo un ResultSet (per le query di lettura) oppure un intero che rappresenta il numero di righe modificate (per le operazioni di scrittura);
- Il Controller consuma il risultato e aggiorna la scena JavaFX corrente (es. ricostruendo dinamicamente le card dei risultati).



Gestione dello stato di sessione

Lo stato dell'utente autenticato è mantenuto interamente in memoria all'interno del Navigator, tramite due campi:

- `idUtenteLoggato`: identificativo numerico dell'utente autenticato (-1 se nessun utente ha effettuato l'accesso), impostato da `LoginController` dopo un login riuscito.
- `ruoloUtenteLoggato`: ruolo dell'utente autenticato (cliente o gestore), utilizzato dai vari controller per abilitare/disabilitare funzionalità riservate e per decidere quale schermata mostrare dopo il login.

Il metodo `Navigator.logout()` azzerà entrambi i campi, riportando l'applicazione allo stato "ospite". Non essendoci un vero canale di rete, non esiste un token di sessione: lo stato vive per l'intera durata del processo JavaFX e viene perso alla chiusura dell'applicazione, comportamento coerente con un'applicazione desktop single-user.

Un ulteriore stato applicativo condiviso è rappresentato dalla classe `SearchState`, che conserva i valori correnti dei filtri di ricerca (città, tipo cucina, range di prezzo, flag delivery/prenotazione, stelle minime) come campi statici, permettendo così a più controller di leggere/scrivere lo stato dei filtri senza doverlo esplicitamente propagare tra le schermate.

Modulo condiviso (DTO)

Le seguenti classi rappresentano i modelli/DTO (Data Transfer Object) condivisi tra i livelli client e server, utilizzati per trasportare i dati letti/scrritti sul database verso l'interfaccia utente e viceversa:

Classe	Descrizione
Ristorante	Modello "di dominio" immutabile di un ristorante (nome, città, nazione, indirizzo, coordinate, fascia di prezzo, tipo di cucina, delivery, prenotazione online), con getter pubblici e nessun setter.
<code>SearchController.RistoranteOggetto</code>	DTO "di presentazione" costruito direttamente a partire da un <code>ResultSet JDBC</code> (costruttore <code>RistoranteOggetto(ResultSet rs)</code>), usato per popolare le card dei risultati di ricerca; include anche i campi calcolati <code>media_stelle</code> (arrotondata a intero in <code>stelleIntere</code>) e le coordinate geografiche.
Utente	Modello di dominio di un utente registrato (nome, cognome, username, email, password, domicilio, data di nascita, tipo account), con getter e setter sui campi modificabili.
<code>SearchState</code>	Contenitore statico dello stato corrente dei filtri di ricerca, condiviso tra le schermate di ricerca (città, tipo cucina, range di prezzo, delivery, prenotazione online, stelle minime) con metodo <code>reset()</code> per riportarlo ai valori di default.

Nota di progetto: il DTO `RistoranteOggetto` è definito come classe interna statica di `SearchController` anziché come classe di primo livello: si tratta di una scelta progettuale che accoppia il modello di presentazione al controller che maggiormente lo utilizza, ma che viene comunque condiviso e riutilizzato da altri controller (ad es. `RestaurantDetailsController` e `Navigator`) tramite il suo nome completo `SearchController.RistoranteOggetto`.



Modulo server

Il "modulo server" comprende tutte le classi del package db, responsabili della connessione al database e dell'esecuzione delle query. Le classi principali sono:

Classe	Responsabilità
DatabaseManager	Gestisce la connessione JDBC a PostgreSQL (host, porta, nome database, credenziali) tramite una singola Connection condivisa (pattern Singleton semplificato); espone initialize(), che verifica/crea il database "theknife" se assente (ensureDatabaseExists()) e delega a DatabaseInitializer la creazione dello schema.
DatabaseInitializer	Verifica l'esistenza delle tabelle applicative; se assenti, esegue lo script theknife_create_db.sql (creazione di tabelle, indici e trigger) e successivamente importa il catalogo iniziale dei ristoranti da un file CSV incluso tra le risorse, con logica di sanificazione dei caratteri speciali (es. simbolo " " usato sia come separatore di campo sia, in alcuni casi, all'interno del nome del locale) e conversione tipizzata dei singoli campi (UUID, double, interi, booleani).
TheKnifeDAO	Data Access Object centrale dell'applicazione: incapsula, tramite PreparedStatement parametriche, tutte le operazioni di lettura e scrittura sul database (autenticazione, ricerca, gestione preferiti, recensioni, risposte, gestione ristoranti lato Gestore).

Schema del database

Lo schema relazionale, definito in theknife_create_db.sql, è composto dalle seguenti tabelle principali:

Tabella	Descrizione sintetica
Utenti	Anagrafica di clienti e gestori, con vincolo CHECK sul campo ruolo ('cliente' o 'gestore') e vincoli geografici sulle coordinate di domicilio.
RistorantiTheKnife	Catalogo dei ristoranti; la chiave primaria id_ristorante è di tipo VARCHAR(50) per ospitare gli UUID provenienti dal dataset CSV di importazione. Include un riferimento opzionale (id_gestore) al proprietario del locale.
Recensioni	Recensioni dei clienti sui ristoranti, con vincolo UNIQUE(id_ristorante, id_utente) che garantisce al massimo una recensione per cliente per locale, e vincolo CHECK sul range delle stelle (1-5).
RisposteRecensioni	Risposte dei gestori alle recensioni, con vincolo UNIQUE(id_recensione) che impedisce più di una risposta per recensione.
Preferiti	Tabella associativa molti-a-molti tra Utenti e RistorantiTheKnife, con chiave primaria composta.



L'integrità referenziale e i vincoli di dominio sono rinforzati da tre trigger PL/pgSQL:

- `tr_check_ruolo_gestore`: impedisce che un ristorante venga associato, come `id_gestore`, a un utente il cui ruolo non sia 'gestore';
- `tr_check_ruolo_cliente`: impedisce che una recensione venga associata a un utente il cui ruolo non sia 'cliente';
- `tr_check_gestore_risposta`: impedisce che un gestore risponda a una recensione relativa a un ristorante che non gli appartiene, confrontando l'`id_gestore` proposto con quello effettivo del ristorante recensito.

Metodi principali esposti da TheKnifeDAO

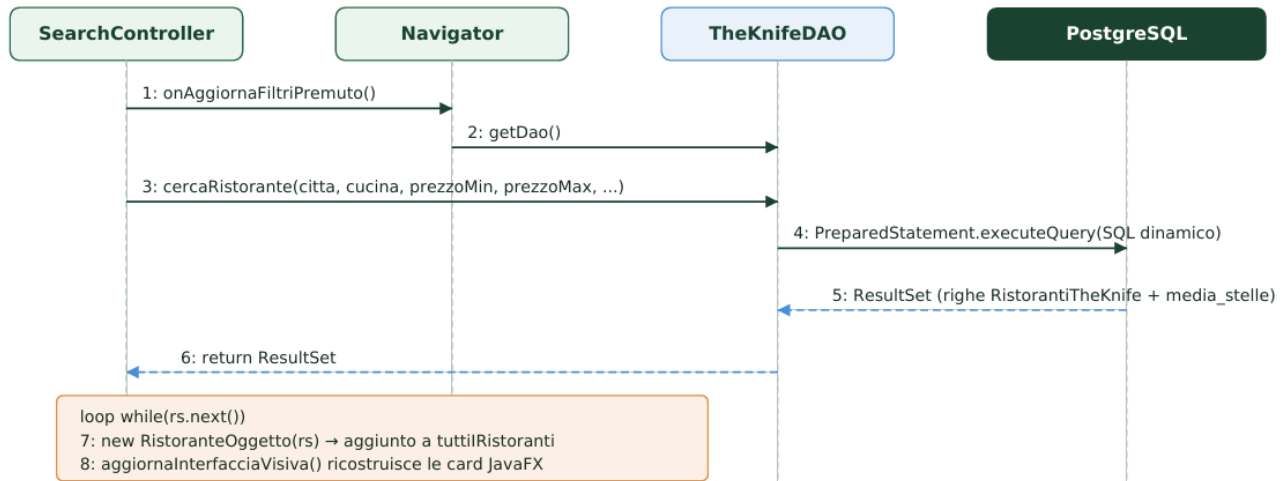
Metodo	Descrizione
<code>registrazione(...)</code>	Inserisce un nuovo utente (cliente o gestore).
<code>login(email)</code>	Recupera id, hash password e ruolo dato l'indirizzo email.
<code>cercaRistorante(...)</code>	Ricerca parametrica con filtri opzionali (città, cucina, range di prezzo, delivery, prenotazione, stelle minime), costruita dinamicamente concatenando clausole WHERE/HAVING solo per i parametri effettivamente valorizzati.
<code>visualizzaRecensioni(idRistorante)</code>	Recupera le recensioni pubbliche di un ristorante con eventuale risposta del gestore.
<code>ristorantiVicini(lat, lon)</code>	Restituisce i 20 ristoranti più vicini a una coppia di coordinate, ordinati per distanza euclidea approssimata.
<code>aggiungiPreferito / rimuoviPreferito</code>	Gestiscono l'aggiunta/rimozione di un ristorante dai preferiti di un cliente.
<code>visualizzaPreferiti(idUtente)</code>	Elenco dei ristoranti preferiti di un cliente con media stelle e conteggio recensioni.
<code>aggiungiRecensione / modificaRecensione / eliminaRecensione</code>	Operazioni CRUD sulle recensioni di un cliente, con verifica di appartenenza tramite <code>id_utente</code> nella clausola WHERE.
<code>mieRecensioni(idUtente)</code>	Elenco delle recensioni scritte da un cliente, con ristorante e risposta del gestore associati.
<code>aggiungiRistorante(...)</code>	Inserisce un nuovo ristorante associandolo al gestore che lo crea.
<code>visualizzaRiepilogo(idGestore)</code>	Riepilogo dei ristoranti di un gestore con media stelle e numero di recensioni.
<code>visualizzaRecensioniGestore(idGestore)</code>	Tutte le recensioni ricevute dai ristoranti di un gestore, con indicazione se già risposte.



Metodo	Descrizione
rispostaRecensione(...)	Inserisce la risposta di un gestore a una recensione, protetta dal trigger tr_check_gestore_risposta.

Diagramma di sequenza Server

Il diagramma seguente illustra il flusso di una tipica operazione di ricerca ristoranti, dal click dell'utente sul pulsante "Aggiorna" fino al rendering dei risultati, evidenziando il ruolo del "modulo server" (TheKnifeDAO e PostgreSQL).



- 1. SearchController: onAggiornaFiltriPremuto()
- 2. SearchController → Navigator: getDao()
- 3. SearchController → TheKnifeDAO: cercaRistorante(citta, cucina, prezzoMin, prezzoMax, ...)
- 4. TheKnifeDAO → PostgreSQL: PreparedStatement.executeQuery(SQL dinamico)
- 5. PostgreSQL → TheKnifeDAO: ResultSet (righe RistorantiTheKnife + media_stelle)
- 6. TheKnifeDAO → SearchController: return ResultSet
- 7. loop while(rs.next()): new RistoranteOggetto(rs) → aggiunto a tuttilRistoranti
- 8. aggiornaInterfacciaVisiva() ricostruisce le card JavaFX
- 9. rendering dei risultati e della paginazione nella VBox containerRisultati



Modulo client

Il "modulo client" comprende le viste FXML, i rispettivi Controller JavaFX e il componente Navigator, responsabile della navigazione tra le schermate. Le classi principali sono:

Classe	Responsabilità
TheKnifeApp	Entry point dell'applicazione (estende javafx.application.Application); inizializza il Navigator con lo Stage principale e carica la prima schermata pubblica.
Navigator	Singleton che centralizza la navigazione tra scene FXML (navigateTo, navigateToSearchWithQuery, navigateToSearchWithAdvancedFilters, navigateToRestaurantDetails, backToSearchResults), lo stato di sessione dell'utente autenticato e il riferimento al DAO condiviso. Implementa inoltre un meccanismo di caching della schermata di ricerca (cachedSearchView), che permette di tornare istantaneamente ai risultati precedenti senza dover ripetere la query.
HomeController / HomeLoggedController	Gestiscono rispettivamente la home page pubblica e quella per utenti autenticati: ricerca globale, scelta rapida per categoria di cucina, pannello filtri avanzati.
LoginController	Gestisce sia il login sia la registrazione di nuovi utenti, smistando l'utente autenticato verso la home cliente o la dashboard gestore in base al ruolo letto dal database.
SearchController	Controller più complesso dell'applicazione: gestisce i filtri di ricerca, l'esecuzione dinamica delle query tramite il DAO, la costruzione manuale (via codice Java) delle card dei risultati e la paginazione lato client dei risultati già recuperati.
RestaurantDetailsController	Popola dinamicamente la scheda di dettaglio di un ristorante a partire da un'istanza di RistoranteOggetto, gestendo anche l'abilitazione condizionale del pulsante di prenotazione.
CustomerProfileController	Gestisce la visualizzazione e la modifica del profilo del cliente autenticato.
OwnerDashboardController	Gestisce la dashboard del Gestore, inclusa la costruzione dinamica (via codice Java) delle card di recensione con relativo form di risposta.

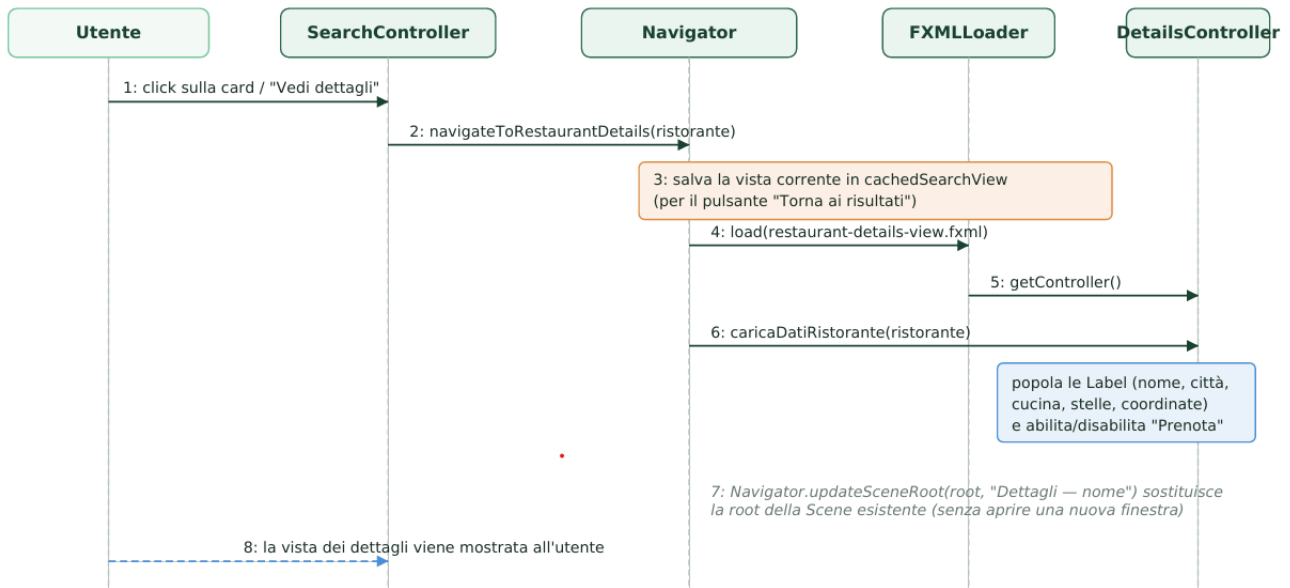
Meccanismo di navigazione

Tutte le transizioni di schermata passano attraverso il Navigator, che utilizza un FXMLLoader per caricare il file FXML richiesto e, quando necessario, recupera il relativo Controller tramite loader.getController() per invocare metodi di inizializzazione personalizzati (ad esempio inizializzaRicercaGlobale(...) o caricaDatiRistorante(...)) prima di mostrare effettivamente la nuova schermata. La sostituzione della schermata avviene tramite scene.setRoot(root) quando una Scene è già stata creata, evitando così di distruggere e ricreare la finestra a ogni navigazione.

Diagramma di sequenza Client

Il diagramma seguente mostra il flusso di navigazione client-side quando un utente, dalla pagina dei risultati di ricerca, clicca su un ristorante per visualizzarne la scheda di dettaglio, includendo il meccanismo di caching della vista di ricerca che consente il successivo ritorno immediato ai risultati.





- 1. Utente → SearchController: click sulla card / "Vedi dettagli"
- 2. SearchController → Navigator: navigateToRestaurantDetails(ristorante)
- 3. Navigator: salva la vista corrente in cachedSearchView (per il pulsante "Torna ai risultati")
- 4. Navigator → FXMLLoader: load(restaurant-details-view.fxml)
- 5. Navigator → FXMLLoader: getController()
- 6. Navigator → DetailsController: caricaDatiRistorante(ristorante) — popola le Label (nome, città, cucina, stelle, coordinate) e abilita/disabilita "Prenota"
- 7. Navigator.updateSceneRoot(root, "Dettagli — nome") sostituisce la root della Scene esistente (senza aprire una nuova finestra)
- 8. la vista dei dettagli viene mostrata all'utente

La stessa logica di caching viene sfruttata dal pulsante "← Torna ai risultati" presente nella schermata di dettaglio: invocando `Navigator.backToSearchResults()`, la root precedentemente salvata in `cachedSearchView` viene ripristinata immediatamente come contenuto della scena corrente, senza rieseguire alcuna query verso il database.

Considerazioni finali: l'architettura descritta, pur non essendo distribuita, adotta comunque una netta separazione delle responsabilità tra presentazione (client), logica di accesso ai dati (server) e modelli condivisi (DTO), rendendo l'applicazione facilmente estendibile: ad esempio, il livello DAO potrebbe in futuro essere esposto dietro un vero endpoint di rete (REST o RMI) senza dover riscrivere la logica dei Controller JavaFX, che già oggi dipendono esclusivamente dall'interfaccia offerta da `TheKnifeDAO` e non direttamente da JDBC o da SQL.



BASE DI DATI

Il sistema utilizza PostgreSQL come Database Management System (DBMS) relazionale. La scelta di un database SQL garantisce l'integrità referenziale e la persistenza robusta.

Entità identificate

TheKnife è una piattaforma per la ricerca e la gestione di ristoranti, ispirata a TheFork. Il sistema prevede tre tipologie di attori: utenti non registrati (*guest*), clienti registrati e gestori di ristoranti registrati.

Entità	Descrizione	Attributi principali
Utente	Persona registrata. Può avere ruolo <i>cliente</i> o <i>gestore</i> (generalizzazione parziale ed esclusiva).	nome, cognome, email, password, data_nascita, domicilio
Cliente	Specializzazione di Utente. Può scrivere recensioni e salvare preferiti.	(eredita da Utente)
Gestore	Specializzazione di Utente. Inserisce e gestisce ristoranti, risponde alle recensioni.	(eredita da Utente)
Ristorante	Locale inserito da un gestore, con caratteristiche geografiche e di servizio.	nome, nazione, città, indirizzo, lat, lon, prezzo, delivery, prenotazione, cucina
Recensione	Valutazione (stelle 1–5 + testo) lasciata da un cliente per un ristorante.	stelle, testo, data
Risposta	Risposta del gestore a una recensione. Al massimo una per recensione. Entità debole dipendente da Recensione.	testo, data

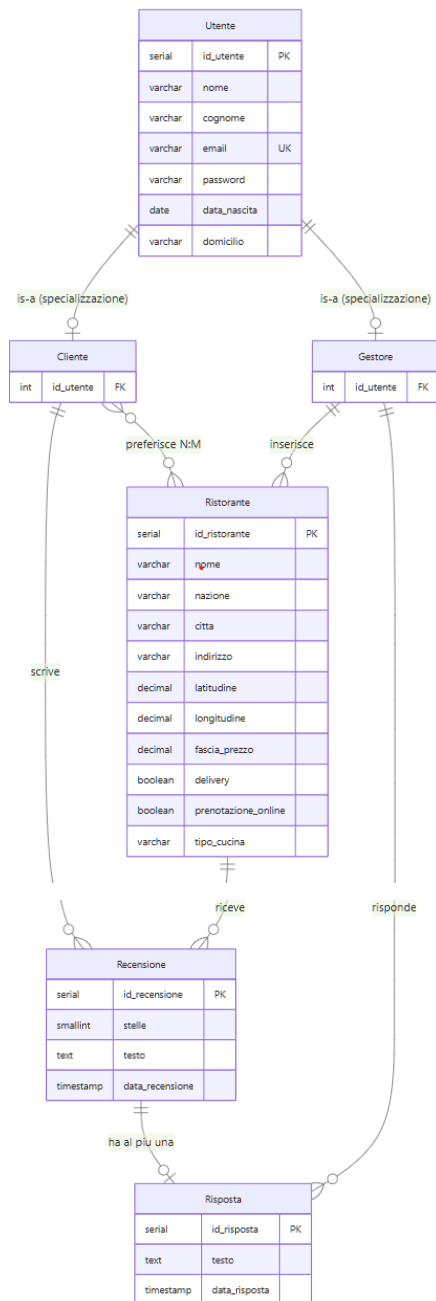
Relazioni e Cardinalità

Associazione	Entità coinvolte	Cardinalità	Note
Inserisce	Gestore → Ristorante	(1,N)	Un gestore inserisce uno o più ristoranti
Scrive	Cliente → Recensione	(1,N)	Un cliente può scrivere più recensioni
Riceve	Ristorante → Recensione	(1,N)	Un ristorante riceve più recensioni
Ha al più una risposta	Recensione → Risposta	(0,1)	Ogni recensione ha al massimo una risposta
Risponde	Gestore → Risposta	(1,N)	Solo il gestore del ristorante recensito risponde
Preferisce	Cliente ↔ Ristorante	(N:M)	Un cliente salva più ristoranti preferiti



Schema Concettuale ER (pre-ristrutturazione)

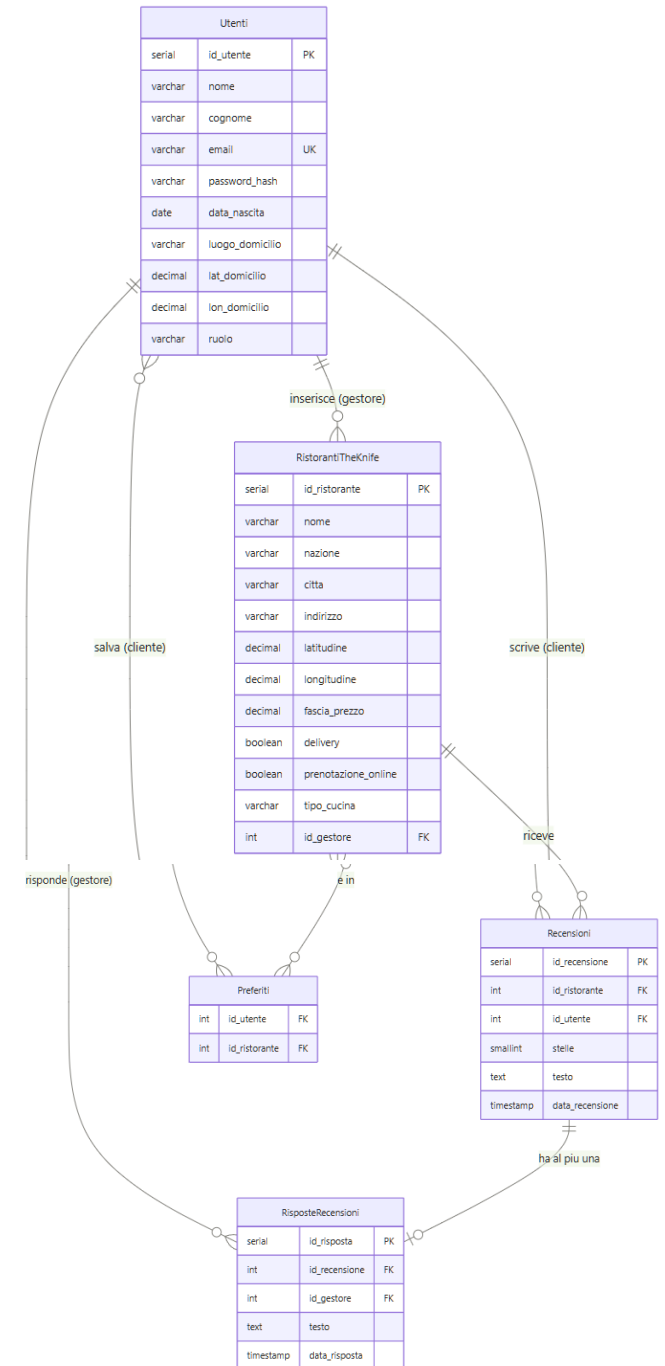
Il diagramma seguente rappresenta lo schema ER **prima della ristrutturazione**. La generalizzazione **Utente** → {**Cliente**, **Gestore**} è ancora esplicitata: le due specializzazioni sono entità distinte che ereditano gli attributi di Utente. La Risposta è modellata come entità debole identificata dalla Recensione cui appartiene.



□ Entità ||--|| Generalizzazione (is-a) ||--o{ Uno a molti ||--o{ Uno a zero/uno }o--o{ Molti a molti

Schema ER (post-ristrutturazione)

Il diagramma seguente rappresenta lo schema ER **dopo la ristrutturazione**. La generalizzazione è stata eliminata (Utenti unica tabella con discriminatore ruolo), RisposteRecensioni è diventata tabella autonoma, la relazione N:M è risolta nella tabella Preferiti.



□ Entità ||--o{ Uno a molti ||--o{ Uno a zero/uno }o--o{ Molti a molti (risolto)

Ristrutturazione dello Schema ER

La ristrutturazione trasforma lo schema ER eliminando generalizzazioni, entità deboli e risolvendo le relazioni N: M.

- **Eliminazione della generalizzazione Utente → {Cliente, Gestore}**

Prima

Superentità **Utente** con due sottotipi distinti: **Cliente** e **Gestore**. Generalizzazione parziale ed esclusiva.

I dati anagrafici di cliente e gestore sono identici. Non esistono attributi specifici per nessuno dei due sottotipi. Il collasso in un'unica tabella con discriminatore evita JOIN inutili durante il login e semplifica la registrazione, che è unica per entrambi i ruoli. La scelta "collasso verso il padre" è preferita rispetto a "collasso verso i figli" per evitare NULL multipli e tabelle ridondanti.

Dopo

Unica tabella **Utenti** con campo discriminatore ruolo ('cliente'/'gestore'). I vincoli sui ruoli sono garantiti dai trigger.

- **Eliminazione dell'entità debole Risposta**

Prima

Risposta era un'entità debole identificata dalla recensione cui appartiene. La cardinalità (0,1) garantiva al massimo una risposta per recensione.

La traduzione di un'entità debole richiede una chiave primaria composta difficile da gestire via JDBC. La tabella autonoma con UNIQUE su **id_recensione** garantisce lo stesso vincolo in modo più chiaro e con migliori prestazioni nelle query.

Dopo

RisposteRecensioni è una tabella autonoma con chiave primaria propria e UNIQUE (**id_recensione**) che garantisce al massimo una risposta.

- **Risoluzione della relazione N:M**

La relazione molti-a-molti tra Cliente e Ristorante è tradotta nella tabella associativa **Preferiti** con chiave primaria composta (**id_utente**, **id_ristorante**). Questa struttura impedisce duplicati e supporta direttamente le operazioni **aggiungiPreferito()**, **rimuoviPreferito()** e **visualizzaPreferiti()**.

Schema Relazionale

Traduzione dello schema ER ristrutturato in schema relazionale.

```
Utenti(id_utente PK, nome, cognome, email UK, password_hash, data_nascita, luogo_domicilio, lat_domicilio, lon_domicilio, ruolo)

RistorantiTheKnife(id_ristorante PK, nome, nazione, citta, indirizzo, latitudine, longitudine, fascia_prezzo, delivery, prenotazione_online, tipo_cucina, id_gestore FK-Utenti)

Recensioni(id_recensione PK, id_ristorante FK-RistorantiTheKnife, id_utente FK-Utenti, stelle, testo, data_recensione,
    UNIQUE(id_ristorante, id_utente))

RisposteRecensioni(id_risposta PK, id_recensione FK-Recensioni UK, id_gestore FK-Utenti, testo, data_risposta)

Preferiti(id_utente PK+FK-Utenti, id_ristorante PK+FK-RistorantiTheKnife)
```



Tabelle dello schema relazionale

Utenti	RistorantiTheKnife	Recensioni	RisposteRecensioni
id_utente PK SERIAL nome VARCHAR(100) cognome VARCHAR(100) email UK VARCHAR(255) password_hash VARCHAR(255) data_nascita OPT DATE luogo_domicilio VARCHAR(255) lat_domicilio OPT DECIMAL(9,6) lon_domicilio OPT DECIMAL(9,6) ruolo VARCHAR(10)	id_ristorante PK SERIAL nome VARCHAR(255) nazione VARCHAR(100) città VARCHAR(100) indirizzo VARCHAR(255) latitudine DECIMAL(9,6) longitudine DECIMAL(9,6) fascia_prezzo DECIMAL(8,2) delivery BOOLEAN prenotazione_online BOOLEAN tipo_cucina VARCHAR(100) id_gestore FK → Utenti	id_recensione PK SERIAL id_ristorante FK → RistorantiTheKnife id_utente FK → Utenti stelle SMALLINT (1-5) testo TEXT data_recensione TIMESTAMP UNIQUE (id_ristorante, id_utente)	id_risposta FK SERIAL id_recensione FK UK → Recensioni id_gestore FK → Utenti testo TEXT data_risposta TIMESTAMP UNIQUE (id_recensione) → max 1 risposta
		Preferiti id_utente FK FK → Utenti id_ristorante FK FK → RistorantiTheKnife PK composta (id_utente, id_ristorante)	

Vincoli e integrità dei dati

Per garantire la robustezza del Data Tier, sono stati implementati i seguenti vincoli:

- **Vincoli Foreign Key (FK)** sono 7 in totale:

Tabella	Campo FK	Riferimento	ON DELETE	ON UPDATE
RistorantiTheKnife	id_gestore	Utenti(id_utente)	RESTRICT	CASCADE
Recensioni	id_ristorante	RistorantiTheKnife(id_ristorante)	CASCADE	CASCADE
Recensioni	id_utente	Utenti(id_utente)	CASCADE	CASCADE
RisposteRecensioni	id_recensione	Recensioni(id_recensione)	CASCADE	CASCADE
RisposteRecensioni	id_gestore	Utenti(id_utente)	RESTRICT	CASCADE
Preferiti	id_utente	Utenti(id_utente)	CASCADE	CASCADE
Preferiti	id_ristorante	RistorantiTheKnife(id_ristorante)	CASCADE	CASCADE

RESTRICT su **id_gestore** in RistorantiTheKnife — se si tenta di eliminare un gestore che ha ristoranti attivi, il DB blocca l'operazione. È una scelta di sicurezza: un ristorante non può restare senza proprietario.

CASCADE su tutto il resto — se si elimina un ristorante, vengono eliminate automaticamente le sue recensioni e le relative risposte. Se si elimina un utente, spariscono le sue recensioni e i suoi preferiti. Mantiene la consistenza del DB senza dati orfani.

- **Unique Constraints** applicate a:
 - **Utenti.email**: Garantisce che ogni utente abbia un'e-mail univoca, che funge da username per il login.
 - **Recensioni (id_ristorante, id_utente)**: Garantisce che un cliente possa scrivere al massimo una recensione per ristorante.
 - **RisposteRecensioni.id_recensione**: Garantisce al massimo una risposta per ogni recensione.
 - **idx_utenti_email**: Indice unico sulla e-mail per rendere veloce il login. Tecnicamente ridondante ma lo abbiamo lasciato per chiarezza documentale.
- **Check Constraints**:
 - Usati in tabella Utenti, RistorantiTheKnife, Recensioni. In totale 7, uno per ogni vincolo di dominio esprimibile direttamente in PostgreSQL.



- **Vincoli procedurali (Trigger)**

Alcuni vincoli non sono esprimibili con CHECK standard in PostgreSQL poiché richiedono sub-query su altre tabelle. Vengono implementati tramite trigger BEFORE INSERT OR UPDATE.

Rif.	Trigger	Tabella	Descrizione
V1	tr_check_ruolo_gestore	RistorantiTheKnife	id_gestore deve riferirsi a un utente con ruolo = 'gestore'
V2	tr_check_ruolo_cliente	Recensioni	id_utente deve riferirsi a un utente con ruolo = 'cliente'
V5	tr_check_gestore_risposta	RisposteRecensioni	Il gestore che risponde deve essere il gestore del ristorante recensito

Utilizzo delle basi di dati nel sistema

L'interazione con PostgreSQL avviene esclusivamente nel Modulo Server attraverso uno strato di astrazione DAO (Data Access Object).

La classe richieste DB centralizza tutta la logica SQL. Utilizza il driver JDBC per stabilire connessioni sicure e gestire il ciclo di vita delle istruzioni:

- **Connection Pooling:** Gestisce l'apertura e la chiusura delle connessioni per ottimizzare le risorse.
- **Prepared Statements:** Tutte le query sono parametrizzate per prevenire attacchi di SQL Injection, garantendo che i dati inseriti dagli utenti non possano alterare la logica del database.

Per mantenere il codice pulito e modulare, le stringhe SQL sono organizzate in una classe dedicata (Queries). Questo permette di:

- Modificare la struttura delle query (es. ottimizzazione di una JOIN) senza dover intervenire nella logica di business del server.
- Mantenere una visione d'insieme di tutti gli endpoint del database, dai filtri di ricerca per autore alla generazione della media voti.

STRUTTURE DATI

Nella scelta delle strutture dati per il nostro **TheKnife** (è una piattaforma per la ricerca di ristoranti), abbiamo cercato il miglior equilibrio possibile tra velocità del software e semplicità di sviluppo.

Componenti Principali:

- **TheKnife** (Main/Controller): Gestisce l'interfaccia utente e il flusso dell'applicazione
- **GestoreRistoranti:** Gestisce la collezione di ristoranti e le operazioni di ricerca
- **GestoreUtenti:** Gestisce la collezione di utenti e l'autenticazione
- **UtenteCorrente:** Mantiene lo stato dell'utente loggato

Entità del Dominio:

- **Utente:** Contiene informazioni personali, credenziali, preferiti e ruolo (Cliente/Ristoratore)
- **Ristorante:** Contiene dati geografici, caratteristiche, recensioni e proprietario
- **Recensione:** Contiene valutazione, testo, data e risposta del ristorante
- **FiltroRicerca:** Oggetto per incapsulare i criteri di ricerca

Organizzazione dei Dati:

- **Liste:** Utilizzate per memorizzare collezioni di ristoranti, utenti e recensioni
- **Associazioni:** Ristorante-Proprietario, Utente-Preferiti, Ristorante-Recensioni



SCELTE ALGORITMICHE

1. Ricerca e Filtraggio

- **Algoritmo:** Ricerca lineare con filtri multipli
- **Complessità:** $O(n)$ dove n è il numero di ristoranti
- **Implementazione:** Scansione sequenziale applicando filtri per tipo cucina, prezzo, servizi

2. Ricerca Geografica

- **Algoritmo:** Confronto per nome città (ricerca per prossimità geografica)
- **Complessità:** $O(n)$ per la ricerca per località
- **Limitazione:** Non implementa calcolo di distanza reale basato su coordinate

3. Autenticazione

- **Algoritmo:** Ricerca lineare nella lista utenti
- **Complessità:** $O(n)$ per ogni tentativo di login
- **Sicurezza:** Hashing delle password tramite MessageDigest

4. Gestione Recensioni

- **Algoritmo:** Ricerca lineare per utente specifico
- **Complessità:** $O(r)$ dove r è il numero di recensioni per ristorante
- **Calcolo media:** Ricalcolo dinamico della media stelle

5. Persistenza Dati

- **Algoritmo:** Serializzazione/deserializzazione completa
- **Complessità:** $O(n)$ per caricamento e salvataggio
- **Approccio:** Caricamento all'avvio, salvataggio alla chiusura

Ottimizzazioni Possibili:

- **HashMap** per ricerche $O(1)$ su ID/username
- **Indici geografici** per ricerche spaziali efficienti
- **Caching** per risultati di ricerca frequenti
- **Lazy loading** per dati non immediatamente necessari

Permette una complessità temporale lineare in cambio di una implementazione diretta e mantenibile.

